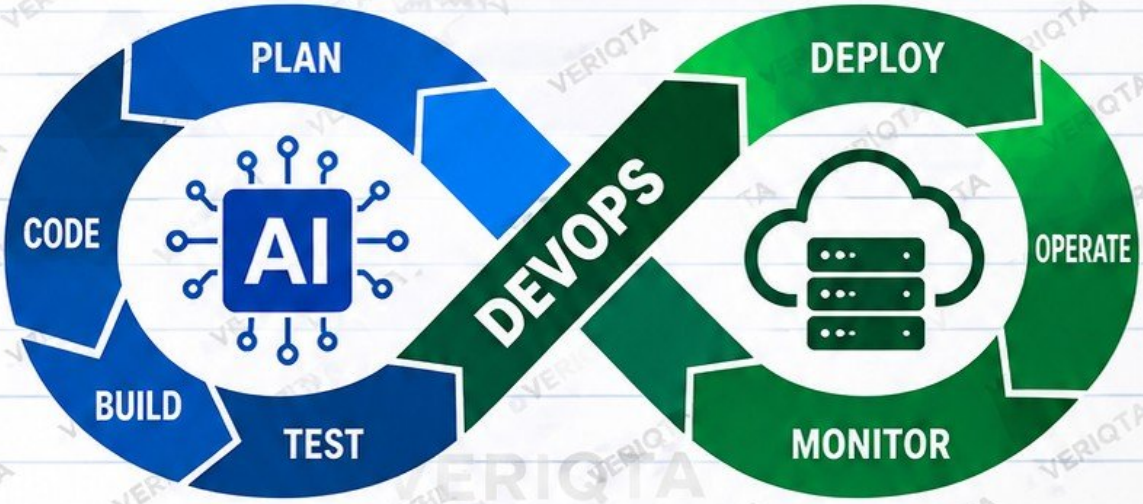
VERIQTA

AI-POWERED DEVOPS PROJECTS HANDBOOK

20 PRACTICAL PROJECTS TO BUILD REAL-WORLD AI-POWERED DEVOPS SKILLS




LINUX


GIT


DOCKER


KUBERNETES

AWS


TERRAFORM


MONITORING

- Step-by-Step Projects
- Real-World DevOps Use Cases
- AI Integration with DevOps Tools
- Troubleshooting and Best Practices
- Beginner-Friendly but Production- Focused

BUILD
AUTOMATE
TROUBLESHOOT
OPERATE
WITH AI

VERIQTA

LEARN TODAY. BUILD TOMORROW.

AI-POWERED DEVOPS
WHAT ARE WE ACTUALLY BUILDING?

1 WHAT IS AI-POWERED DEVOPS?

- AI-powered DevOps uses artificial intelligence to assist, accelerate and improve DevOps work.
- It combines DevOps tools with AI (LLMs, APIs, and intelligent agents) to automate analysis, provide insights, and help solve problems faster.
- It does not replace engineers. It makes engineers more effective.

2 AI ASSISTANCE VS TRADITIONAL AUTOMATION

TRADITIONAL AUTOMATION

- Follows fixed rules
- Does exactly what you configure
- No reasoning or context
- Good for repeated tasks
- Examples: shell scripts, CI/CD pipelines, IaC

AI ASSISTANCE

- Understands context
- Analyzes and explains
- Provides recommendations
- Helps troubleshoot
- Adapts to different scenarios
- Examples: LLMs, AI agents, AI assistants integrated with DevOps tools

3 WHERE AI FITS IN THE DEVOPS LIFECYCLE

AI can assist in every stage:

- **Plan** – analyze requirements, suggest designs
- **Code** – code review, explain code, suggest improvements
- **Build** – analyze build failures
- **Test** – generate test cases, analyze test results
- **Deploy** – review configurations, detect risks
- **Operate** – troubleshoot, automate repetitive tasks
- **Monitor** – analyze metrics, logs, and alerts

4 KEY CONCEPTS EXPLAINED SIMPLY

LLM (Large Language Model)	An AI model that understands and generates human-like text. Examples: ChatGPT, Claude, Gemini.
API (Application Programming Interface)	A way for your application to communicate with an AI model or other tools.
AI Agent	An AI system that can use tools, make decisions, and complete tasks based on a goal.
Tools	DevOps tools such as Git, Docker, Kubernetes, Terraform, cloud APIs, monitoring tools, etc.
Context	The information you give the AI (e.g. logs, code, error messages, configuration files) so it can provide accurate responses.

5 WHAT AI SHOULD AND SHOULD NOT CONTROL

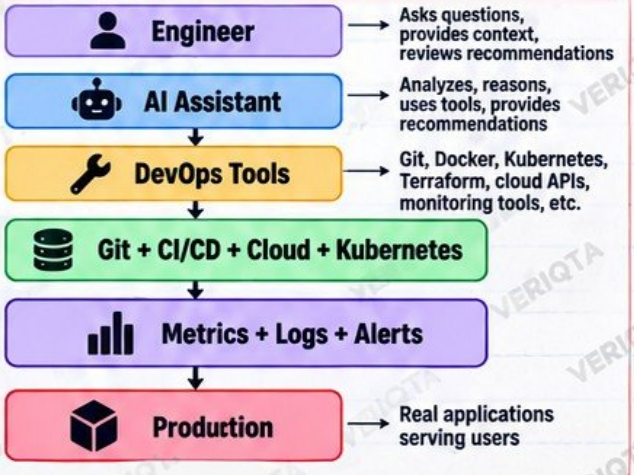
AI CAN HELP WITH

- Analyzing logs and errors
- Explaining commands and code
- Suggesting fixes and best practices
- Reviewing configurations
- Generating documentation
- Detecting risks and anomalies
- Automating safe, low-risk tasks

AI SHOULD NOT DIRECTLY

- Make production changes without approval
- Delete critical resources
- Modify security or IAM policies
- Approve deployments on its own
- Access sensitive data without proper controls
- Replace human judgment

6 HIGH-LEVEL ARCHITECTURE



7 PRODUCTION RULE

"AI RECOMMENDS FIRST, HUMANS APPROVE HIGH-IMPACT ACTIONS."

AI is a powerful assistant, but you are the engineer. Always validate, review, and take responsibility.

BUILD YOUR AI DEVOPS LAB

SET UP THE ENVIRONMENT USED THROUGHOUT THE HANDBOOK.

1 WHAT WE ARE BUILDING

In this lab, you will set up a complete environment that we will use for all projects in this handbook. You will install the required tools, configure access, and test everything works.

This is a one-time setup. Once completed, you can follow the next projects without repeating these steps.

2 TOOLS TO SET UP

- Git repository
- Python environment
- API access
- .env configuration
- GitHub repository
- Docker
- Linux terminal
- JSON files
- YAML files

3 KEY CONCEPTS WE WILL USE

API Keys	Used to access LLM services such as OpenAI. Keep them private.
Environment Variables	Store sensitive information outside your code using .env.
Dependencies	Install required Python packages (such as requests).
Secrets Protection	Do not hardcode credentials. Use .env and .gitignore.
.gitignore	Tells Git which files and folders to exclude from your repository.

4 QUICK SETUP COMMANDS

Create a project folder

```
mkdir ai-devops-lab
cd ai-devops-lab
```

Create a Python virtual environment

```
python3 -m venv venv
source venv/bin/activate
```

Install required packages

```
pip install requests python-dotenv
```

Create a .env file

```
touch .env
```

Initialize a Git repository

```
git init
```

5 EXAMPLE .ENV FILE

```
OPENAI_API_KEY=sk-your-api-key-here
MODEL=gpt-4o
# Add other variables as needed
```

6 IMPORTANT: .GITIGNORE

```
# Python
venv/
__pycache__/
.env
# IDE files
.vscode/
# OS files
.DS_Store
```

7 FIRST PROJECT: ASK AN LLM A DEVOPS QUESTION

1. Write a simple Python script.
2. Send a DevOps question to an LLM through the API.
3. Receive and print the response.
4. Verify it works.

Example question:
"Explain what a Kubernetes Deployment is and when to use it."

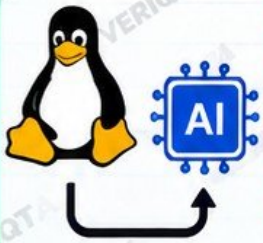
8 NEXT STEPS

- Ensure all tools are working
- Confirm API access
- Test your Python script
- Keep your API key secure
- You are now ready for the next project!

PROJECT 1: AI LINUX COMMAND EXPLAINER

BUILD AN ASSISTANT THAT RECEIVES A LINUX COMMAND AND EXPLAINS IT

1 WHAT WE ARE BUILDING



- You enter a Linux command.
- The AI explains what it does.
- It shows important flags.
- It tells you the expected effect.
- It highlights potential risks.
- It suggests safer alternatives.
- It helps you learn and avoid mistakes.

2 HOW IT WORKS

User enters command → AI Assistant → Analysis and explanation → Safe and useful response

The AI uses context and best practices to explain the command. It does not execute the command.

3 WHAT THE AI EXPLAINS

What it does	Simple explanation of the command.
Important flags	Key options and their purpose.
Expected effect	What will happen when you run it.
Potential risk	Possible risks or side effects.
Safer alternatives	A safer way to achieve the same goal.

4 EXAMPLE

INPUT (User command)

```
sudo systemctl restart nginx
```

AI OUTPUT (Explanation)

```
Action:      Restarts Nginx
Impact:      Brief service interruption
             possible
Risk:        Medium
Verify:      systemctl status nginx
Alternative: Try systemctl reload nginx
             for zero-downtime (if supported)
```

5 SAMPLE OUTPUT FORMAT

```
Command: <your command>
What it does: <explanation>
Important flags: <flags and meaning>
Expected effect: <what will happen>
Potential risk: <risks and side effects>
Safer alternative: <recommended option>
Verify: <how to confirm it worked>
```

6 PROTECTION AGAINST DANGEROUS COMMANDS

- Detects destructive commands (rm -rf /, dd, :(){ :|:& };:, etc).
- Warns the user before explaining high-risk commands.
- Clearly explains the potential damage.
- Suggests safer alternatives.
- Never encourages running destructive commands.
- Reminds the user to understand the command before running it.

"The AI should educate, not enable dangerous actions."

7 TRY IT YOURSELF

1. Write a simple Python script.
2. Send a Linux command to an LLM through the API.
3. Display the structured explanation.
4. Test different commands (safe ones).
5. Add safety checks for destructive commands.

Example commands to try:

- ls -l
- df -h
- systemctl status nginx
- kubectl get pods

8 KEY TAKEAWAYS

- AI helps you understand Linux commands.
- It explains, it does not execute.
- Always review the output and think critically.
- Use safer alternatives when available.
- Never run a command you do not understand.

PROJECT 2: AI LINUX LOG ANALYZER

GIVE LOGS TO THE AI AND GET A CLEAR EXPLANATION

1 WHAT WE ARE BUILDING

Build a Python-based assistant that analyzes application or Linux logs using an LLM. It identifies errors, warnings, important timestamps, likely failure, evidence, and recommended investigation steps.

This helps you troubleshoot faster and understand what is happening in your systems.

2 ARCHITECTURE

Application
↓
Logs
↓
Python Parser
↓
LLM API
↓
Failure Summary
↓
Suggested Investigation

3 WHAT THE AI IDENTIFIES

- **Errors** — Finds error messages and their meaning.
- **Warnings** — Identifies warnings that may indicate a problem.
- **Important timestamps** — Highlights when key events happened.
- **Likely failure** — Suggests the most likely cause.
- **Evidence** — Shows relevant log lines that support the analysis.
- **Recommended investigation** — Provides clear next steps for troubleshooting.

4 SAMPLE LOG INPUT

```
2024-05-14 10:12:01 INFO  Starting nginx...
2024-05-14 10:12:03 ERROR Failed to bind to
  port 80: Address already in use
2024-05-14 10:12:03 WARN  Retrying in 5 seconds
2024-05-14 10:12:08 ERROR Still cannot bind
  to port 80
2024-05-14 10:12:08 INFO  Service stopped
```

5 SAMPLE AI OUTPUT

Summary
Nginx failed to start because port 80 is already in use.

Key Findings
- Error: Failed to bind to port 80
- Timestamp: 2024-05-14 10:12:03
- Service stopped after multiple retries

Likely Cause
Another process is using port 80.

Recommended Investigation
- Check which process is using port 80
  `sudo lsof -i :80`
- Stop the conflicting process if not needed
  `sudo kill <PID>`
- Verify Nginx status
  `sudo systemctl status nginx`

6 WHY AI DIAGNOSIS IS A HYPOTHESIS

- AI analyzes patterns in the logs and makes an educated guess.
- It does not have real-time access to your environment.
- It can miss important context.
- It may be wrong or incomplete.
- Always verify the AI suggestions using real commands and evidence.
- Use AI as a starting point, not a final answer.

"AI gives you a hypothesis, not proof."

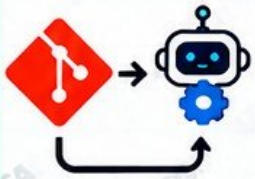
7 BEST PRACTICES

- Provide relevant log lines (not too much, not too little).
- Include timestamps and context.
- Verify all findings with real commands.
- Combine AI analysis with your own knowledge.
- Use AI to save time, but make the final decision.
- Keep sensitive data out of logs.
- Continuously learn from real incidents.

PROJECT 3: AI GIT COMMIT AND PULL REQUEST ASSISTANT

TURN YOUR GIT CHANGES INTO CLEAR, USEFUL INSIGHTS

1 WHAT WE ARE BUILDING



Build a tool that analyzes your Git changes and uses AI to generate commit messages, pull request summaries, find potential risks, recommend tests, and create a reviewer checklist.

We use git diff as controlled context so the AI only sees the relevant changes.

2 WHAT IT GENERATES

- **Commit-message suggestion**
  Clear and meaningful commit message.
- **Pull request summary**
  Summary of what changed and why.
- **Files changed**
  List of modified, added, and deleted files.
- **Potential risks**
  Highlights risky changes.
- **Testing recommendations**
  Suggested tests to run.
- **Reviewer checklist**
  Key points for code review.

3 HOW IT WORKS

4 EXAMPLE: GIT DIFF AS INPUT

```
# Show changes to send to AI
git diff
# Or compare with main branch
git diff main...HEAD
```

5 SAMPLE AI OUTPUT

Commit Message Suggestion:
`Fix: handle missing environment variable in user authentication`

Pull Request Summary:
This change adds validation for the `USER_TOKEN` environment variable and returns a clear error message when it is missing.

Files Changed:
- app/auth.py (modified)
- app/utils.py (modified)
- tests/test_auth.py (added)

Potential Risks:
- May affect existing authentication flow

Testing Recommendations:
- Run unit tests for auth module
- Test with and without `USER_TOKEN`

Reviewer Checklist:
- ☑ Code logic is correct
- ☑ Environment variables handled properly
- ☑ Tests added and passing
- ☑ No sensitive data exposed
- ☑ Ready for deployment

6 SECURITY BEST PRACTICES

- ✓ Never send secrets, API keys, or tokens.
- ✓ Use git diff to send only relevant changes.
- ✓ Exclude sensitive files (e.g. .env, config files).
- ✓ Use .gitignore to prevent accidental exposure.
- ✓ Review AI output before using it.
- ✓ Do not allow AI to access your full repository unless necessary.

7 IMPORTANT REMINDER

Never send secrets or sensitive repository data to an AI service unnecessarily.

Use controlled context, remove sensitive information, and always review the output.

8 NEXT STEPS

1. Build the Python script.
2. Integrate with an LLM API.
3. Test with real Git changes.
4. Improve the output with more rules.
5. Add it to your daily workflow.

PROJECT 4: AI YAML CONFIGURATION REVIEWER

USE AI TO FIND ISSUES AND IMPROVE YOUR CONFIGURATIONS

1 WHAT WE ARE BUILDING

Build an AI reviewer that analyzes YAML configuration files and provides clear, practical feedback. It works with:

- Kubernetes YAML
- GitHub Actions
- Docker Compose
- CI/CD configuration

You provide a YAML file and the AI reviews it for errors, risks, and best practices.

2 HOW IT WORKS

The AI analyzes the YAML content, checks for common issues, explains the findings, and suggests improvements.

3 WHAT THE AI IDENTIFIES

Syntax concerns	Invalid YAML, wrong structure, indentation issues
Missing configuration	Required fields not set
Security risks	Exposed secrets, overly permissive permissions, unsafe settings
Reliability problems	Misconfigurations that can cause failures or downtime
Suggested improvements	Best practices and optimized configuration examples

4 EXAMPLE: KUBERNETES YAML REVIEW

INPUT (deployment.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: app
        image: nginx:latest
        ports:
        - containerPort: 80
```

AI OUTPUT (review)

```
Findings:
• Uses latest image tag (not recommended)
• No resource requests and limits
• No liveness or readiness probes
• No security context
Risk Level: Medium
Suggested Improvements:
• Use a specific image tag
• Add resource limits
• Add health probes
• Add security context
```

5 EXAMPLE: GITHUB ACTIONS REVIEW

INPUT (.github/workflows/ci.yml)

```yaml
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v3
    - name: Build
      run: docker build .
```

AI OUTPUT (review)

```
Findings:
• No branch filter (runs on all branches)
• No dependency caching
• No security scanning
Risk Level: Low
Suggested Improvements:
• Add branch filters
• Enable caching
• Add security scan step
• Use pinned action versions
```

6 BEST PRACTICES

- Keep YAML simple and readable
- Use specific version tags (avoid latest)
- Add resource requests and limits
- Include health checks (probes)
- Follow least privilege principles
- Validate YAML with proper tools
- Review AI suggestions before applying
- Test in a non-production environment

7 IMPORTANT LESSON

Always validate AI-generated YAML with deterministic tools before deployment. AI is helpful, but it can make mistakes. Use tools like kubectl, yamllint, docker compose config, or actionlint to confirm the configuration is correct.

8 VALIDATION COMMANDS

```
# Kubernetes YAML
kubectl apply --dry-run=client
-f file.yaml

# YAML syntax check
yamllint file.yaml
```

```
# Docker Compose
docker compose config

# GitHub Actions
python -m actionlint file.yaml
```

PROJECT 5: AI DOCKERFILE REVIEWER

ANALYZE YOUR DOCKERFILE AND BUILD MORE SECURE, EFFICIENT IMAGES

1 WHAT WE ARE BUILDING

Create a Dockerfile analysis tool that checks your Dockerfile and provides clear, practical feedback using static checks and AI. It helps you build smaller, more secure, and production-ready images.

The tool identifies common issues, explains the risks, and suggests improvements.

2 WHAT WE CHECK

Large base images	Suggests smaller, official images (e.g. alpine).
Root containers	Warns if running as root. Suggests non-root user.
Secrets	Detects hardcoded secrets and sensitive data.
Layer inefficiency	Finds unnecessary layers and suggests combining.
Dependency practices	Checks package installation best practices.
Missing health checks	Warns if no HEALTHCHECK is defined.
Unnecessary packages	Identifies unused packages to reduce image size.
Image security concerns	Checks for vulnerable or outdated components.

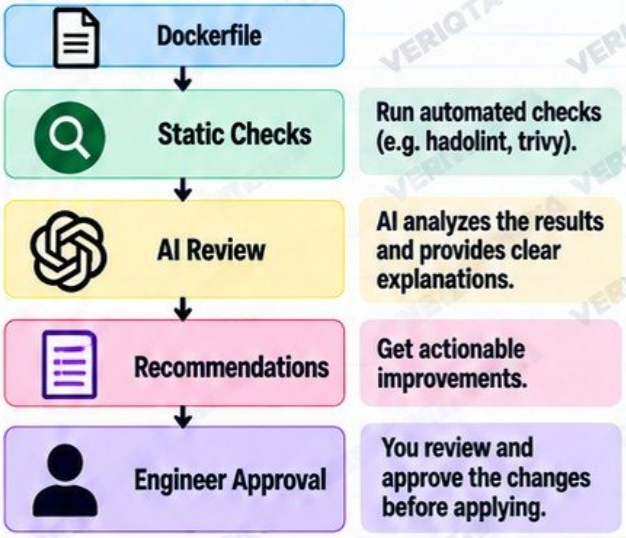
3 EXAMPLE DOCKERFILE (WITH ISSUES)

```
1  FROM ubuntu:latest
2  RUN apt-get update && apt-get install -y \
3      python3 pip git curl
4  COPY . /app
5  WORKDIR /app
6  RUN pip install -r requirements.txt
7  EXPOSE 8000
8  CMD ["python3", "app.py"]
```

4 AI REVIEW OUTPUT (EXAMPLE)

```
Findings:
1. Large base image: ubuntu:latest (use a smaller image)
2. Running as root (create a non-root user)
3. No .dockerignore (may include unnecessary files)
4. Missing HEALTHCHECK
5. Multiple layers can be combined
6. No specific version for packages
Risk Level: Medium
Suggested Improvements:
  • Use python:3.11-slim
  • Add a non-root user
  • Add HEALTHCHECK
  • Use .dockerignore
  • Combine RUN commands
  • Pin package versions
```

5 ANALYSIS FLOW



6 IMPORTANT SECURITY LESSON

Never send secrets or sensitive repository data to an AI service unnecessarily.

Use .dockerignore, remove sensitive information, and only share the relevant parts of your Dockerfile.

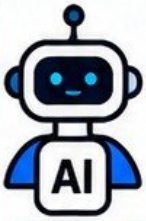
7 BEST PRACTICES

- Use official and minimal base images (e.g. alpine, slim)
- Run as a non-root user
- Add a HEALTHCHECK
- Use .dockerignore
- Combine layers and clean up packages
- Pin package versions
- Scan images with security tools
- Always review AI suggestions before applying

PROJECT 6: AI CI/CD FAILURE INVESTIGATOR

TURN FAILED PIPELINES INTO CLEAR ANSWERS

1 WHAT WE ARE BUILDING



Build a tool that takes failed CI/CD pipeline information and uses AI to analyze the failure, identify likely causes, and suggest clear next steps. It helps you troubleshoot faster and learn from failures.

You provide the pipeline details and the AI investigates, explains, and guides you to a fix.

2 INPUTS TO THE AI

- **Job name** (e.g. build, test, deploy)
- **Exit code** (e.g. 1, 2, 126)
- **Failed step** (e.g. npm install, unit tests)
- **Relevant logs** (paste error logs or key lines)
- **Recent change** (e.g. code change, dependency update, config change)

3 AI OUTPUT FLOW

SYMPTOM
What is happening?

↓

EVIDENCE
What in the logs shows it?

↓

LIKELY CAUSES
What could be the reason?

↓

NEXT CHECK
What should you verify next?

↓

POSSIBLE FIX
How to fix it?

↓

VERIFICATION
How to confirm it's resolved?

4 EXAMPLE INPUT (FAILED PIPELINE)

```
Job name: build
Exit code: 1
Failed step: npm install
Recent change: Updated package.json
Logs:
npm ERR! code E404
npm ERR! 404 Not Found - GET
npm ERR! 404 'my-private-package@1.0.0'
npm ERR! 404
npm ERR! 404 'my-private-package@1.0.0' is
 not in this registry.
```

5 EXAMPLE AI OUTPUT

```
SYMPTOM:
Pipeline failed during npm install
with a 404 error.

EVIDENCE:
Log shows package not found in
registry.

LIKELY CAUSES:
• Private package not published
• Wrong registry URL
• Incorrect package name or version
• Missing authentication token

NEXT CHECK:
Verify registry configuration
and package version.

POSSIBLE FIX:
Update .npmrc with correct registry
and ensure the package exists.

VERIFICATION:
Re-run the pipeline and confirm
npm install completes successfully.
```

6 COMMON CI/CD FAILURE AREAS

- **Build Failures** — Dependencies, build tools, syntax errors
- **Test Failures** — Unit tests, integration tests, flaky tests
- **Container Build Failures** — Dockerfile, base image, missing packages
- **Deployment Failures** — Kubernetes, Helm, infrastructure issues
- **Permission Errors** — Secrets, tokens, IAM, access control
- **External Service Failures** — Databases, APIs, package registries

7 CORRELATION vs ROOT CAUSE

CORRELATION

- Two events happen at the same time.
- One event may seem related to the other.
- Does not necessarily mean one caused the other.
- Example: A deployment and a service outage happened at the same time.

ROOT CAUSE

- The actual underlying reason for the failure.
- Explains why the problem happened.
- Requires evidence, not assumptions.
- Example: The deployment introduced a breaking change in a configuration file, which caused the service to fail.

8 BEST PRACTICES

- Provide relevant logs (not too much, not too little).
- Include the recent changes.
- Use AI as an assistant, not a final answer.
- Verify suggestions before making changes.
- Learn from each failure and document it.
- Keep sensitive information out of the input.

9 IMPORTANT SECURITY LESSON

Never send secrets or sensitive repository data to an AI service unnecessarily.

Remove tokens, passwords, keys, and private information from logs. Only share the minimum required context.

PROJECT 7: AI PULL REQUEST RISK ANALYZER

ANALYZE CHANGES. UNDERSTAND RISK. DEPLOY SAFER.

1 WHAT WE ARE BUILDING

Create an AI system that analyzes pull request changes, identifies the type of changes, classifies the deployment risk, and provides clear reasons and recommendations.

The tool helps you understand the impact of changes before they are deployed to production.

2 TYPES OF CHANGES TO ANALYZE

- **Infrastructure** (e.g. Terraform, CloudFormation)
- **Database migrations** (e.g. schema changes)
- **IAM** (e.g. policies, roles, permissions)
- **Networking** (e.g. VPC, security groups, ingress/egress)
- **Kubernetes** (e.g. manifests, Helm charts)
- **Dependencies** (e.g. package versions)
- **Application configuration** (e.g. env files, config maps)

3 HOW IT WORKS

Pull Request (Git diff) → Change Analysis → AI Risk Analysis → Risk Report and Recommendations

4 EXAMPLE INPUT (GIT DIFF)

```
diff --git a/iam/policy.json b/iam/policy.json
@@ -1,5 +1,6 @@
{
  "Version": "2012-10-17",
  "Statement": [
+     { "Effect": "Allow", "Action": "s3:*", "Resource": "*" }
  ]
}
```

5 EXAMPLE AI OUTPUT

```
Risk: HIGH
Reason: IAM policy modified (broad permissions)
Recommendations:
    • Security review
    • Staging validation
    • Rollback plan
```

6 RISK CLASSIFICATION

LOW RISK

- Documentation changes
- Comments
- Minor code changes
- Non-production configs

Usually safe to deploy with standard checks.

MEDIUM RISK

- Application config changes
- Dependency updates
- Non-critical infra changes
- Kubernetes manifest updates

Requires additional review and testing.

HIGH RISK

- IAM changes
- Database migrations
- Networking changes
- Production infrastructure
- Security-related changes
- Breaking changes

Requires security review, staging validation, and rollback plan.

7 EXAMPLE RISK ANALYSIS RESULT

```
Risk: HIGH
Reason: IAM policy modified
Details:
  • Pull request adds s3:* permission.
  • This provides full access to all S3 resources.
  • This can lead to data exposure or accidental deletion.
Recommended Actions:
  • Security review by IAM team
  • Validate in staging environment
  • Implement least privilege permissions
  • Prepare rollback plan
```

HIGH RISK PR

AI advises. It does not automatically approve production changes.

8 BEST PRACTICES

- Always review AI findings, do not blindly trust them.
- Keep the context focused (use git diff , not entire repo).
- Classify the impact on security, reliability, and cost.
- Use AI as a second pair of eyes.
- Require human approval for all production changes.
- Combine AI analysis with automated checks (tests, security scans).
- Maintain a rollback plan for high-risk changes.

9 IMPORTANT LESSON

AI advises. It does not automatically approve production changes.

Use AI to identify risk, but always apply human judgment, organizational policies, and proper review processes.

PROJECT 8: AI TERRAFORM PLAN REVIEWER

ANALYZE INFRASTRUCTURE CHANGES BEFORE YOU APPLY

1 WHAT WE ARE BUILDING

Build an AI assistant that analyzes terraform plan output, identifies what will change, explains the impact, highlights risks and costs, and gives clear recommendations before you run terraform apply.

The tool helps you make safer, smarter infrastructure decisions with AI, but you always review and approve the changes.

2 INPUT: TERRAFORM PLAN

```
# Generate a plan
terraform plan -out=tfplan

# Show plan as text
terraform show -no-color tfplan
```

The AI reads the plan output (text or JSON) and analyzes the changes.

3 WHAT WE ANALYZE

- **Resources created** — New infrastructure resources
- **Resources modified** — Existing resources being changed
- **Resources destroyed** — Resources that will be removed
- **Security implications** — IAM, security groups, public access
- **Cost-sensitive changes** — Resources that may increase cost
- **Network exposure** — Internet-facing or public endpoints
- **High-risk modifications** — Critical infrastructure changes

4 EXAMPLE TERRAFORM PLAN (SNIPPET)

```
# Terraform will perform the following actions:
+ aws_instance.web
    ami             = "ami-0abcd1234"
    instance_type   = "t3.medium"
    tags            = { Name = "web-server" }

~ aws_security_group.web_sg
    ingress         = (known after apply)
    egress          = (known after apply)

- aws_s3_bucket.logs
    bucket          = "prod-logs"
    force_destroy   = true
```

5 AI OUTPUT (EXAMPLE)

```
Terraform Plan Analysis
Summary:
+ Resources to create: 1
~ Resources to modify: 1
- Resources to destroy: 1
Details:
1. aws_instance.web (create)
   - Low risk
   - Estimated monthly cost: $30
2. aws_security_group.web_sg (modify)
   - Medium risk
   - Check for open ingress rules
3. aws_s3_bucket.logs (destroy)
   - HIGH RISK
   - This will permanently delete
     the S3 bucket and all data!
Recommendations:
- Review security group changes
- Confirm S3 bucket deletion is intended
- Verify cost implications
- Run in a non-production environment
  first
```

6 RISK LEVELS

LOW RISK
- Minor changes
- Tag updates
- Non-critical resources
- No security impact

MEDIUM RISK
- Configuration changes
- Security group updates
- Instance type changes
- Potential cost increase
- Needs review

HIGH RISK
- Resource destruction
- IAM changes
- Network exposure
- Database changes
- Production resources
- Requires approval

7 WORKFLOW

1. **terraform plan** — Generate plan output
2. **Extract plan data** (text or JSON)
3. **AI Analysis** — Identify risks and explain
4. **Recommendations** — Clear next steps
5. **Engineer Approval** — Review and decide
6. **terraform apply** — Only after approval

8 IMPORTANT LESSON

Pay special attention to destructive actions.

Resources marked for deletion (-) can lead to permanent data loss. Always review AI suggestions, validate with terraform plan, and never apply changes blindly.

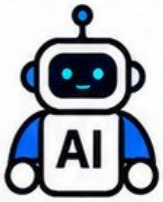
9 BEST PRACTICES

- Use terraform plan with -out and analyze the saved plan
- Review AI findings, do not blindly trust them
- Check for security, cost, and networking impact
- Always validate in a non-production environment
- Use policy as code (e.g. Sentinel, OPA) for additional guardrails
- Maintain human approval for all production changes

PROJECT 9: AI INFRASTRUCTURE TROUBLESHOOTING ASSISTANT

FIND PROBLEMS FASTER. FOLLOW A STRUCTURED APPROACH. GET CLEAR ANSWERS.

1 WHAT WE ARE BUILDING



Build a structured troubleshooting assistant for cloud infrastructure that helps you diagnose issues such as application downtime, ALB unhealthy targets, and other infrastructure problems.

The assistant guides you through a step-by-step investigation, asks for evidence, and provides clear explanations and next steps.

2 EXAMPLE INPUT

Application unavailable

ALB reports unhealthy targets

Environment: Production
Region: us-east-1
Service: web application
Symptom: Users cannot access the application

3 INVESTIGATION FLOW

DNS Check domain resolution

Load Balancer
Check ALB status and target health

Security Groups
Verify inbound and outbound rules

Network
Check subnets, route tables, NACLs

Compute
Verify EC2/ECS/EKS instance health

Application
Check service, ports and configuration

Logs
Review application, system and infrastructure logs

4 EXAMPLE AI RESPONSE (SNIPPET)

```
Investigation: ALB Unhealthy Targets
1. DNS
   Status: OK
   Evidence: Domain resolves to ALB DNS name
2. Load Balancer
   Status: Unhealthy targets
   Evidence: 2/2 targets failing health checks
   Next: Check target group health check settings
3. Security Groups
   Status: Needs verification
   Evidence required: Inbound rules on port 80/443
4. Network
   Status: Pending
   Action: Verify subnet route tables and NACLs
...
Conclusion: Not enough evidence to determine
root cause. Please provide the security group
rules and target instance logs.
```

5 WHAT THE AI DOES

Asks for evidence
Requests relevant data before making conclusions

Guides step-by-step
Follows a structured troubleshooting flow

Analyzes information
Finds patterns and possible causes

Suggests next steps
Provides clear and practical actions

Explains reasoning
Tells you why each step matters

Avoids false conclusions
Does not assume a root cause without evidence

6 COMMON CHECKS AT EACH LAYER

Layer	What to Check	Example Commands / Tools
DNS	Domain resolves correctly	nslookup, dig
Load Balancer	Listener, target health	aws elbv2 describe-target-health
Security Groups	Inbound/outbound rules	aws ec2 describe-security-groups
Network	Subnets, routes, NACLs	aws ec2 describe-route-tables
Compute	Instance/pod health	aws ec2 describe-instances kubectl get pods
Application	Service status, config	systemctl status, docker ps
Logs	Application and system logs	journalctl, tail, CloudWatch

7 BEST PRACTICES

- Provide complete and accurate information
- Share relevant logs and error messages
- Follow the troubleshooting flow in order
- Verify each layer before moving to the next
- Use AI as an assistant, not a replacement
- Document findings and solutions
- Confirm the fix with real tests
- Learn from each incident and improve runbooks

8 IMPORTANT LESSON

AI must request evidence before declaring a root cause.

Similarity does not always mean causation. Understand the difference between correlation and root cause, and always validate with real data.

PROJECT 10: AI KUBERNETES MANIFEST REVIEWER

CATCH MISTAKES EARLY. DEPLOY SAFER. RUN MORE RELIABLE WORKLOADS.

1 WHAT WE ARE BUILDING

Build an AI assistant that reviews Kubernetes manifests, finds configuration issues, explains problems, and suggests improvements before workloads reach the cluster.

The tool combines AI reasoning with schema validation and policy checks to give accurate, practical and beginner-friendly feedback.

2 MANIFESTS WE ANALYZE

- **Deployments** — Replicas, strategy, labels
- **Services** — Type, ports, selectors
- **ConfigMaps** — Configuration data
- **Secrets** — Sensitive data handling
- **Resource requests and limits** — CPU and memory
- **Probes** — Liveness, readiness, startup
- **Security contexts** — Run as non-root, capabilities
- **Image configuration** — Image tags, pull policy, registry

3 COMMON BEGINNER MISTAKES DETECTED

- Missing or incorrect resource requests and limits
- Using latest image tag
- Exposing services unnecessarily
- Incorrect service type
- Missing probes
- Running containers as root
- Hardcoded secrets in manifests
- Incorrect ConfigMap references
- Missing labels and selectors
- Using privileged containers
- No security context settings
- Invalid or deprecated API versions

4 EXAMPLE MANIFEST (WITH ISSUES)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: web
        image: nginx:latest
        ports:
        - containerPort: 80
```

ISSUES

- Using latest image tag
- No resource requests and limits
- No probes
- No security context
- Missing labels (best practices)

5 AI REVIEW OUTPUT (EXAMPLE)

```
Resource: Deployment/web-app
Overall Risk: MEDIUM
Findings:
1. Using latest image tag
   - Risk: Unpredictable deployments
   - Recommendation: Use a specific version tag (e.g. nginx:1.25)
2. Missing resource requests and limits
   - Risk: Pod can consume too much CPU/memory
   - Recommendation: Add requests and limits
3. No liveness or readiness probes
   - Risk: Unhealthy pods may not be detected
   - Recommendation: Add HTTP or TCP probes
4. No security context
   - Risk: Container may run as root
   - Recommendation: Set runAsNonRoot: true
```

6 TOOLS TO COMBINE WITH AI

- **kubeval** — Schema validation
- **kubeconform** — Validate against Kubernetes schemas
- **OPA (Open Policy Agent)** — Policy checks
- **Kyverno** — Security and best practice policies
- **LLM (e.g. GPT)** — Explain issues and suggest improvements

7 IMPROVED MANIFEST (AI SUGGESTION)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      securityContext:
        runAsNonRoot: true
      containers:
      - name: web
        image: nginx:1.25
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"
        livenessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 10
          periodSeconds: 10
```

8 BEST PRACTICES

- Always use specific image tags
- Set resource requests and limits
- Configure liveness and readiness probes
- Run containers as non-root
- Use security contexts
- Manage secrets securely (no hardcoded values)
- Validate manifests with kubeval or kubeconform
- Enforce policies with OPA or Kyverno
- Review AI suggestions, do not blindly trust them
- Test in a non-production environment first

9 IMPORTANT LESSON

AI helps you find problems, but you are still responsible.

Review the suggestions, validate with tools, and use your engineering judgment. Never deploy to production based only on AI output.

PROJECT 11: AI KUBERNETES TROUBLESHOOTING ASSISTANT

FIND ISSUES FASTER. USE REAL EVIDENCE. GET CLEAR EXPLANATIONS.

1 WHAT WE ARE BUILDING

Build an AI assistant that helps you investigate real Kubernetes failures using actual cluster data, explains the likely causes, and provides clear next steps.

The assistant uses evidence from kubectl commands to analyze problems. It does not guess or invent cluster state.

2 COMMON KUBERNETES FAILURES

- **CrashLoopBackOff** — Container keeps crashing
- **ImagePullBackOff** — Cannot pull the container image
- **Pending Pods** — Waiting for resources or scheduling
- **OOMKilled** — Container killed due to out of memory
- **Failed probes** — Liveness or readiness probe failing
- **Service unreachable** — Cannot access the application

3 TROUBLESHOOTING FLOW

1. **Identify the problem** (from pod status, logs or alerts)
2. **Gather real evidence** (use kubectl commands)
3. **Share the evidence with AI** (logs, events, describe output)
4. **Analyze and explain** (likely causes and reasoning)
5. **Get next steps** (specific and actionable)
6. **Verify the fix** (confirm the application works)

4 EXAMPLE: POD IN CRASHLOOPBACKOFF

```
$ kubectl get pods
NAME                  READY   STATUS             RESTARTS   AGE
web-app-7d8f7b6d9-abc12   0/1     CrashLoopBackOff   5          3m

$ kubectl describe pod web-app-7d8f7b6d9-abc12
...
Events:
  Type     Reason     Age    From     Message
  ----     ------     ----   ----     -------
  Warning  BackOff    1m     kubelet  Back-off restarting
                                      failed container

$ kubectl logs web-app-7d8f7b6d9-abc12
Error: Cannot connect to database
at startup...
```

5 AI ANALYSIS OUTPUT (EXAMPLE)

```
Issue: CrashLoopBackOff
Summary: Container is crashing because it cannot
connect to the database.

Evidence:
- Logs show connection failure
- Pod restarts multiple times
- Environment variable DB_HOST is missing

Likely Cause:
Application cannot reach the database due to
missing or incorrect configuration.

Next Steps:
1. Check ConfigMap or Secret for DB_HOST
2. Verify database service is reachable
3. Confirm network policies and security groups
4. Fix configuration and redeploy

Note: This analysis is based only on the provided
kubectl output.
```

6 USEFUL KUBECTL COMMANDS

```
kubectl get pods
List all pods and their status

kubectl describe pod <pod-name>
Show detailed information and events

kubectl logs <pod-name>
View container logs

kubectl get events
--sort-by=.metedata.creationTimestamp
Check recent cluster events

kubectl get svc
Check service configuration

kubectl get endpoints
Verify service endpoints

kubectl top pods
Check resource usage

kubectl get nodes
Check node status
```

7 BEST PRACTICES

- Always provide real command output.
- Include relevant logs and events.
- Share only the necessary information.
- Be specific about what you are seeing.
- Verify AI suggestions before making changes.
- Do not allow AI to guess or invent cluster state.
- Understand the reasoning behind each step.
- Use AI as a troubleshooting partner, not a replacement for your own judgment.

8 IMPORTANT LESSON

AI can help you troubleshoot, but it must not hallucinate cluster state.

Always provide real evidence from your cluster. Validate the AI's suggestions. You are responsible for making the final decision.

9 REAL-WORLD SCENARIOS

- Application not starting
- Database connection issues
- Service accessible internally but not externally
- Permission or RBAC errors
- Node or resource capacity problems
- Configuration and environment issues

PROJECT 12: AI OBSERVABILITY INVESTIGATOR

TURN TELEMETRY INTO ANSWERS. INVESTIGATE FASTER. RESOLVE SMARTER.

1 WHAT WE ARE BUILDING

Build an AI assistant that combines metrics, logs, traces, and deployment events to investigate incidents, find patterns, generate hypotheses, and guide engineers to the next steps.

The assistant correlates data across telemetry sources to help you understand what is happening, why it might be happening, and what to check next.

2 TELEMETRY SOURCES

Metrics
(e.g. Prometheus, CloudWatch, Datadog, Dynatrace)
CPU, memory, latency, error rates

Logs
(e.g. ELK, Loki, CloudWatch Logs)
Application and infrastructure logs

Traces
(e.g. OpenTelemetry, Jaeger, Dynatrace, AWS X-Ray)
Request flow and service dependencies

Deployment Events
(e.g. Kubernetes, Jenkins, GitHub, Argo CD)
New releases, configuration changes

3 HOW IT WORKS

Metrics · Logs · Traces · Deployment Events

↓

AI
(Analysis and Correlation)

↓

Incident Hypothesis
(Explanation and Evidence)

↓

Next Steps
(What to Inspect and Verify)

4 EXAMPLE SCENARIO

ALERT: High API Latency

```
Service:        payment-service
Time:           2024-08-10 14:32 UTC
Symptom:        Increased latency and 5xx errors
Recent Change:  New deployment 14:20 UTC
```

5 AI ANALYSIS EXAMPLE

Observation:

- Latency increased from 200ms to 2s
- Error rate increased to 5%
- New deployment at 14:20 UTC
- Traces show timeout to database

Hypothesis:

The new deployment introduced a database connection issue causing increased latency and 5xx errors.

Evidence:

- Metrics: Higher DB latency and connection count
- Logs: Connection timeout errors
- Traces: Spans show DB calls taking > 2s
- Events: Deployment of v2.1.0 at 14:20 UTC

Next Steps:

1. Check database health and connection limits
2. Review recent code changes in v2.1.0
3. Compare configuration and environment variables
4. Rollback if necessary

6 KEY QUESTIONS AI SHOULD ASK

What changed?
(releases, config, infra, traffic)

When did degradation begin?
Compare with deployments and events

Which service is affected?
Identify blast radius

What evidence supports the hypothesis?
Show relevant metrics, logs, traces

What should the engineer inspect next?
Provide clear, actionable steps

7 CORRELATION EXAMPLE

Metrics
Latency ↑
Error rate ↑

Logs
Database timeout errors

Traces
Slow DB spans
Service dependency issues

Deployment Events
New version deployed at 14:20 UTC

↓

AI Correlation
Deployment caused DB connection exhaustion, leading to high latency and 5xx errors.

8 BEST PRACTICES

- Combine multiple telemetry sources
- Use real data, not assumptions
- Validate AI findings with dashboards and logs
- Look for correlations, not single data points
- Document findings and resolution steps
- Use AI as a technical assistant, not an authority
- Build runbooks from common incidents

9 IMPORTANT LESSON

AI can find patterns, but it must not hallucinate.

Always provide real telemetry data. Verify the AI's analysis with your own investigation. You are responsible for the final diagnosis and resolution.

10 TOOLS YOU CAN USE

- Prometheus (metrics)
- ELK / OpenSearch (logs)
- Jaeger / OpenTelemetry (traces)
- Kubernetes (events)
- Dynatrace (full observability)

PROJECT 13: AI ALERT TRIAGE SYSTEM

TURN ALERTS INTO ACTION. FASTER INVESTIGATION. CLEARER DECISIONS.

1 WHAT WE ARE BUILDING

Build an AI assistant that receives monitoring alerts, understands the context, enriches the alert with relevant data, and provides clear triage information and recommended next steps.

The goal is to help engineers respond faster, reduce noise, and make better decisions, not to suppress alerts without safeguards.

2 ALERT INPUT (EXAMPLE)

```
Alert: High CPU Usage
Source: Prometheus
Service: api-service
Severity: Warning
Threshold: CPU > 80% for 5m
Time: 2024-08-10 14:32 UTC
Environment: production
Cluster: eks-prod
Labels: { team: backend,
          service: api-service }
Annotations: Runbook: link
```

3 AI TRIAGE OUTPUT (EXAMPLE)

```
Severity: HIGH
Affected Service: api-service
Potential Customer Impact:
Possible slow responses or
timeouts for end users.
Related Signals:
  • CPU sustained at 92% (5m)
  • Increased request latency
  • More 5xx errors in logs
  • Recent deployment 14:20 UTC
Recommended Investigation:
1. Check recent deployment changes
2. Review pod resource usage
3. Inspect application logs
4. Verify downstream dependencies
Escalation Requirement:
Yes – Page on-call engineer
```

4 WHAT THE AI DETERMINES

- **Severity** — Critical / High / Medium / Low
- **Affected service** — Which service or component
- **Potential customer impact** — User-facing impact and business risk
- **Related signals** — Metrics, logs, traces, events, recent changes
- **Recommended investigation** — Clear next steps for engineers
- **Escalation requirement** — Is on-call or leadership notification needed

5 ALERT TRIAGE FLOW

1. **Receive Alert** (from monitoring system)
2. **Enrich with Context** (metrics, logs, traces, events, deployment history)
3. **AI Analysis** (determine severity, impact, related signals)
4. **Recommendations** (provide investigation steps)
5. **Escalation Decision** (decide if on-call notification is required)

6 ALERT ENRICHMENT EXAMPLES

- **Metric context** — CPU, memory, latency, error rates
- **Log analysis** — Error messages, stack traces
- **Trace correlation** — Find related services and requests
- **Deployment events** — Recent releases, config changes
- **Dependency health** — Databases, queues, external APIs
- **Infrastructure context** — Node, pod, cluster, region status
- **Historical patterns** — Similar past incidents and outcomes

7 BEST PRACTICES

- Enrich alerts with relevant context
- Do not suppress alerts automatically
- Validate AI findings with actual data
- Use clear and actionable recommendations
- Include runbook links
- Set clear escalation rules
- Learn from past incidents
- Continuously improve with feedback

8 IMPORTANT LESSON

AI should enrich alerts, not hide problems.

Never let AI close or suppress alerts without human review and safeguards. Use AI to add context, find patterns, and guide faster investigation, while keeping engineers in control.

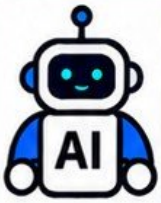
9 REAL-WORLD ALERT EXAMPLES

- High CPU usage on application pods
- High error rate (5xx) on API gateway
- Database connection failures
- Pod repeatedly restarting
- Disk space running low
- Increase in latency for user requests
- External service dependency failures
- Sudden drop in traffic
- Security-related alerts (e.g. unusual access)

PROJECT 14: AI INCIDENT RESPONSE COPILOT

FASTER RESPONSE. CLEARER INVESTIGATION. STRONGER RESOLUTION.

1 WHAT WE ARE BUILDING



Build an AI incident response assistant that helps during outages by collecting evidence, creating a timeline, tracking hypotheses, suggesting actions, and generating a final summary.

The assistant supports engineers with structured investigation, clear context, and actionable guidance while keeping humans in control.

2 INCIDENT RESPONSE FLOW

- **Incident detected** (Alert or user report)
- ↓
- **Evidence collected** (Logs, metrics, traces, events)
- ↓
- **Timeline generated** (Chronological view)
- ↓
- **Hypotheses tracked** (Possible root causes)
- ↓
- **Actions recorded** (What was tried and result)
- ↓
- **Service restored** (Validation and monitoring)
- ↓
- **Incident summarized** (Findings, lessons, next steps)

3 KEY CAPABILITIES

- Collect and analyze evidence from multiple sources
- Correlate metrics, logs, traces and deployment events
- Suggest possible causes with clear reasoning
- Recommend next actions (step-by-step guidance)
- Require human approval for risky operations
- Maintain audit trails for all decisions and actions
- Keep within safe operational boundaries (no autonomous production changes)

4 EXAMPLE INCIDENT TIMELINE

Time (UTC)	Event
10:00	Alert triggered (High error rate)
10:02	AI collected logs, metrics, traces
10:04	Timeline generated
10:06	Hypothesis 1: Database connection issue
10:08	Checked DB metrics (high latency)
10:12	Hypothesis 2: Recent deployment change
10:15	Rolled back deployment (human approval)
10:18	Service recovery confirmed
10:20	Incident summary generated

5 AI OUTPUT EXAMPLE

```
Incident: API Service Outage
Severity: HIGH
Summary:
 Increased 5xx errors starting at
 10:00 UTC. Database latency elevated
 after recent deployment.
Likely Cause:
 Connection pool exhaustion due to
 new configuration.
Recommended Action:
 1. Verify database metrics
 2. Check recent deployment changes
 3. Consider rollback (requires approval)
 4. Monitor for recovery
```

6 HUMAN APPROVAL & SAFETY

- Require human approval for production changes
- Show clear explanation before any action is taken
- Follow least privilege access (no unnecessary permissions)
- Do not make destructive changes automatically
- Allow engineers to review, edit, or reject suggestions
- Operate within defined safe boundaries

7 BEST PRACTICES

- Use real data from your environment
- Combine metrics, logs, traces and events
- Keep humans in the loop
- Record all actions and decisions
- Provide clear and explainable recommendations
- Validate findings before taking action
- Learn from past incidents
- Turn incidents into runbooks and improvements

8 IMPORTANT LESSON

AI can help you respond faster, but humans are still responsible.

Use AI to augment your judgment, not replace it. Always verify the evidence, require approval for critical actions, and maintain a complete audit trail.

9 REAL-WORLD USE CASES

- API service outage
- Database performance issues
- Kubernetes pod failures
- Load balancer errors
- Service degradation after deployment
- Infrastructure failures (e.g. EC2, RDS)
- Multi-service incidents

PROJECT 15: AI ROOT CAUSE AND POSTMORTEM ASSISTANT

TURN INCIDENT DATA INTO CLEAR INSIGHTS. LEARN. IMPROVE. PREVENT REPEAT INCIDENTS.

1 WHAT WE ARE BUILDING

Build an AI assistant that uses incident evidence to generate a first postmortem draft, helping engineers document what happened, why it happened, and how to prevent it in the future.

The assistant organizes real data into a structured postmortem. It does not replace human analysis or accountability.

2 INPUTS (INCIDENT EVIDENCE)

- **Logs** (Application, system, infrastructure)
- **Metrics** (Alerts, dashboards, graphs)
- **Traces** (Request flow, latency, errors)
- **Deployment events** (Recent changes, releases)
- **Incident tickets** (Alerts, on-call notes, communications)
- **Chat/War room logs** (Investigation discussions)
- **Runbooks and documentation** (Context and expected behavior)

3 POSTMORTEM SECTIONS

1. **Incident summary** — What happened?
2. **Customer impact** — Who was affected and how?
3. **Timeline** — Key events in chronological order
4. **Detection** — How was the issue detected?
5. **Technical cause** — What actually caused the issue?
6. **Contributing factors** — What made it worse or easier to happen?
7. **Recovery** — How was the service restored?
8. **Corrective actions** — What will we do to prevent recurrence?

4 EXAMPLE AI POSTMORTEM DRAFT

```
Incident Postmortem (AI Draft)

1. Incident Summary
On Aug 10, 2024, the API service experienced
increased error rates, leading to partial outage.

2. Customer Impact
Approximately 25% of users experienced failures
when creating new accounts for 42 minutes.

3. Timeline (UTC)
10:02   Alert triggered (5xx errors > 5%)
10:05   On-call engineer notified
10:10   Initial investigation started
10:18   Root cause identified
10:22   Fix deployed
10:44   Service fully restored

4. Detection
Detected via Prometheus alert on API error rate.

5. Technical Cause
A misconfigured database connection pool caused
connection exhaustion.

6. Contributing Factors
Recent deployment increased traffic.
Insufficient connection limits.
Lack of load testing.

7. Recovery
Increased DB connection limits and restarted service.

8. Corrective Actions
- Add automated load testing to CI/CD
- Set connection pool alerts
- Improve deployment validation
- Update runbook with troubleshooting steps
```

5 AI WORKFLOW

1. **Collect incident evidence** (logs, metrics, traces, tickets)
2. **Analyze and organize information** (identify key events and patterns)
3. **Generate structured draft** (using defined postmortem sections)
4. **Highlight uncertainties** (flag missing or unclear information)
5. **Suggest corrective actions** (based on evidence and best practices)
6. **Human review and edit** (validate, add context, approve)
7. **Publish and share** (final postmortem)

6 IMPORTANT LESSON

AI must not invent missing events or falsely assign root cause.

The AI should clearly indicate when information is missing or uncertain. Always verify with real evidence and human judgment before finalizing the postmortem.

7 BEST PRACTICES

- Use real incident data, not assumptions
- Be transparent about what is unknown
- Clearly separate facts from hypotheses
- Include both technical and process factors
- Suggest specific, actionable improvements
- Keep postmortems blameless and constructive
- Use consistent templates
- Continuously improve based on past incidents

8 REAL-WORLD BENEFITS

- Faster postmortem creation
- More accurate and complete analysis
- Better learning from incidents
- Stronger preventive actions
- Improved reliability and trust

9 KEY TAKEAWAY

A good postmortem is not about blame.
It is about learning and building a more reliable system.

10 TOOLS YOU CAN USE

OpenAI GPT · Elastic (ELK) · Grafana · Datadog · Dynatrace

PROJECT 16: AI DEVSECOPS SECURITY REVIEWER

SECURE PIPELINES. SAFER INFRASTRUCTURE. SMARTER DECISIONS.

1 WHAT WE ARE BUILDING

Build a security-focused assistant that analyzes findings from security scanners, explains the risks in simple terms, prioritizes them, and suggests remediation steps for DevSecOps environments.

The assistant works with existing security tools. It does not replace scanners. It helps engineers understand, prioritize, and fix issues faster.

2 AREAS OF FOCUS

- **IaC** (Terraform, CloudFormation)
- **Containers** (Docker, container images)
- **Kubernetes** (manifests, configurations)
- **CI/CD** (pipelines, build configurations)
- **IAM** (policies, permissions, access)
- **Secrets** (hardcoded secrets, secret management)
- **Dependencies** (open source libraries, packages)

3 SECURITY REVIEW PIPELINE

Code
(Source code, IaC, manifests, configs)
↓
Security Scanners
(SAST, DAST, container, dependency, IaC, secrets, policy checks)
↓
Findings
(Vulnerabilities, misconfigurations, policy violations)
↓
AI Explanation + Prioritization
(Explain, group, rank by severity and impact)
↓
Engineer Review
(Validate, discuss, decide)
↓
Remediation
(Fix issues and verify)

4 EXAMPLE FINDINGS AND AI ANALYSIS

```
Finding: Hardcoded AWS credentials
Tool: TruffleHog (Secrets Scan)
Severity: HIGH
AI Explanation:
AWS credentials are exposed in the code.
If this repository is public, attackers can
gain full access to your AWS account.
Recommendation:
1. Remove the credentials from the code
2. Rotate the exposed keys
3. Use AWS Secrets Manager or environment variables
---------------------------------------------
Finding: Container running as root
Tool: Trivy (Container Scan)
Severity: MEDIUM
AI Explanation:
Running containers as root increases the risk
of container escape and privilege escalation.
Recommendation:
1. Set a non-root user in the Dockerfile
2. Use securityContext in Kubernetes
3. Apply the principle of least privilege
---------------------------------------------
Finding: Unpinned dependency version
Tool: Snyk (Dependency Scan)
Severity: MEDIUM
AI Explanation:
Using unpinned versions can lead to
unexpected and vulnerable updates.
Recommendation:
1. Use specific version numbers
2. Enable dependency scanning in CI/CD
3. Regularly update and review dependencies
```

5 AI HELPS WITH

- Explaining security findings in simple terms
- Prioritizing by real risk and business impact
- Grouping related issues
- Providing clear remediation steps
- Mapping findings to frameworks (e.g. CIS, NIST, OWASP)
- Reducing false positives through context
- Suggesting best practices
- Generating reports for teams

6 IMPORTANT NOTES

- AI must not replace security scanners.
- AI explains and prioritizes deterministic findings.
- Always validate AI suggestions before taking action.
- Keep human approval for all security changes.
- Use AI within defined security and operational boundaries.

7 BEST PRACTICES

- Use multiple security scanners
- Provide clear context to the AI assistant
- Include environment and business impact
- Verify findings and suggested fixes
- Track and measure remediation progress
- Integrate into CI/CD pipelines
- Follow security frameworks and policies
- Continuously improve with feedback

8 REAL-WORLD TOOLS

- Trivy (Container, IaC, Kubernetes)
- Snyk (Dependencies)
- Checkov (IaC)
- OWASP ZAP (Web security)
- GitHub Advanced Security
- SonarQube (Code quality & security)

9 KEY TAKEAWAY

AI can help you fix security issues faster, but human judgment, validation and secure processes are still essential.

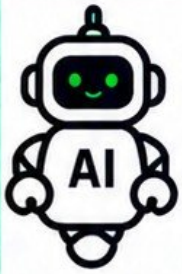
10 COMMON SECURITY RISKS

Exposed secrets · Misconfigurations · Vulnerable dependencies · Insecure containers · Overly permissive IAM · Unsecured infrastructure

PROJECT 17: BUILD A GUARDAILED DEVOPS AI AGENT

CONTROLLED TOOL USE. SAFER ACTIONS. REAL PRODUCTION VALUE.

1 WHAT WE ARE BUILDING



Build a guardrailed DevOps AI agent that can safely use real tools to help engineers investigate, troubleshoot, and operate infrastructure, with strong safety controls and human approval for production changes.

The agent can observe, analyze, and suggest actions, but production-changing actions require explicit human approval.

2 WHAT THE AGENT CAN DO (SAFE)

- **Read logs** (e.g. application, system, cluster)
- **Inspect deployment status** (e.g. Kubernetes, CI/CD)
- **Query monitoring data** (e.g. metrics, alerts, dashboards)
- **Read Kubernetes state** (pods, services, events, resources)
- **Analyze Git changes** (commits, pull requests, diffs)
- **Suggest commands** (safe, read-only by default)

3 ACTIONS AND APPROVAL

Read-Only Actions (Auto Allowed)

- Read logs and metrics
- Query system and cluster state
- Analyze code and configurations
- Generate recommendations
- Explain findings

Production-Changing Actions (Require Explicit Approval)

- Deploy or rollback
- Scale resources
- Modify configurations
- Restart services
- Change infrastructure (IaC)

4 DEVOPS AI AGENT WORKFLOW

1. **OBSERVE** — Collect data from tools and environment
2. **REASON** — Analyze data, identify patterns and possible causes
3. **PROPOSE** — Suggest commands or actions with explanation
4. **APPROVE** — Human reviews and approves (for risky actions)
5. **ACT** — Execute approved action through controlled tools
6. **VERIFY** — Check results and confirm resolution
7. **AUDIT** — Record all actions, decisions and outcomes

5 IMPORTANT SAFETY CONTROLS

- **Allowlists** — Only approved tools and commands
- **Least privilege** — Minimal access and permissions
- **Timeouts** — Prevent long-running or stuck actions
- **Credentials** — Use secure secrets management (no hardcoded credentials)
- **Audit logs** — Log all actions, prompts and outputs
- **Prompt injection protection** — Detect and block malicious instructions
- **Blast-radius control** — Limit scope of changes (namespace, environment, resources)

6 EXAMPLE TOOL INTEGRATIONS

- **Kubernetes** (kubectl - read-only by default)
- **AWS CLI** (read-only by default)
- **Grafana** (query metrics and alerts)
- **ELK / OpenSearch** (read logs)
- **GitHub** (read repositories and PRs)
- **Terraform** (plan and read state)
- **Jenkins** (check build and deployment status)

7 BEST PRACTICES

- Start with read-only access
- Use allowlists, not open-ended access
- Enforce least privilege
- Require human approval for risky actions
- Log everything for audit and accountability
- Test in non-production first
- Continuously review and improve guardrails

8 KEY TAKEAWAY

A DevOps AI agent is most valuable when it is powerful AND safe.

The goal is not to replace engineers, but to give them a trusted assistant that accelerates investigation and operations while keeping systems secure and under control.

9 REAL-WORLD BENEFITS

- Faster troubleshooting
- Reduced mean time to resolution (MTTR)
- More consistent and accurate operations
- Safer automation with human oversight
- Better visibility and audit trails
- Lower operational risk
- More productive engineering teams

PROJECT 18: FINAL PROJECT

AI-POWERED DEVOPS OPERATIONS PLATFORM

BRINGING EVERYTHING TOGETHER. FROM CODE TO PRODUCTION. INTELLIGENT OPERATIONS AT SCALE.

1 PROJECT OVERVIEW

Build a complete AI-powered DevOps operations platform that integrates all the concepts from this handbook into a realistic, end-to-end system for building, deploying, monitoring, and operating applications at scale.

The platform uses an AI DevOps Copilot with context, guardrails, and real tools to help engineers work faster, safer, and smarter.

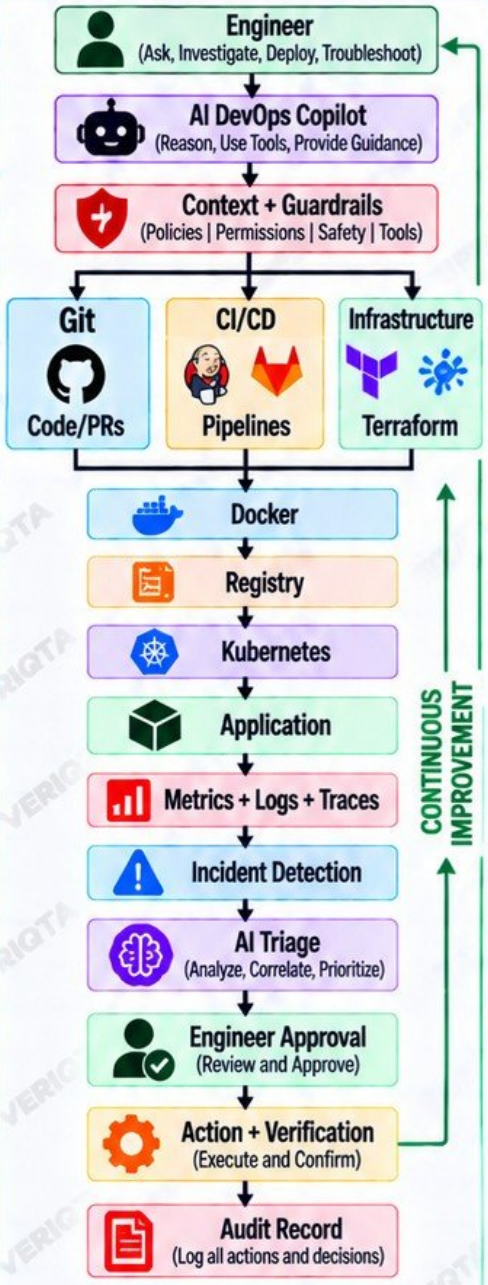
2 KEY GOALS

- Integrate all DevOps components
- Use AI for intelligent assistance
- Implement strong security and guardrails
- Support real-world production workflows
- Improve reliability and faster incident response
- Provide end-to-end observability
- Create a repeatable and scalable platform

3 TECHNOLOGIES USED

- **Git / GitHub** (source control)
- **CI/CD** (GitHub Actions / Jenkins)
- **Terraform** (infrastructure as code)
- **Docker** (containerization)
- **Registry** (container registry)
- **Kubernetes** (orchestration)
- **Prometheus + Grafana** (monitoring)
- **ELK / OpenSearch** (logging)
- **OpenTelemetry** (tracing)
- **AI** (LLMs + tools)
- **Security tools** (SAST, DAST, IaC scan)
- **Cloud** (AWS / Azure / GCP)

4 PLATFORM ARCHITECTURE



5 AI COPILOT CAPABILITIES

- Read logs and analyze issues
- Inspect deployment status
- Query monitoring data
- Read Kubernetes state
- Analyze Git changes
- Suggest commands and next steps
- Explain risks and best practices
- Use tools within defined permissions
- Require human approval for production changes

6 IMPORTANT SAFETY CONTROLS

- Tool allowlists (approved commands only)
- Least privilege access
- Timeouts for long-running actions
- Secure credentials (no hardcoded secrets)
- Prompt injection protection
- Blast-radius control (limit scope of changes)
- Audit logs (all actions recorded)
- Human approval for production changes

7 BUSINESS VALUE

- Faster and safer deployments
- Reduced downtime (MTTR)
- More reliable and consistent operations
- Improved efficiency and lower costs
- Better engineer productivity
- Real-world, production-ready skills

8 KEY TAKEAWAY

AI does not replace DevOps engineers. It empowers them to operate faster, safer, and more effectively when combined with the right tools, guardrails, and engineering discipline.

9 NEXT STEPS

- Deploy the platform in a lab environment
- Test real scenarios (deploy, break, fix)
- Iterate and improve
- Document your learnings
- Build your own production-ready version

10 YOU DID IT!

You have completed the AI for DevOps Projects Handbook. You now have the knowledge and practical projects to build, operate, and improve real-world DevOps environments with AI.
Keep learning. Keep building.